

AI Video Generation System - Project Proposal and Technical Specification

1. Executive Summary

This proposal outlines the development of an advanced, AI-powered content generation pipeline capable of transforming a topic and character input into high-quality multimedia content, including long-form narrated audio, talking face videos, and fully edited, YouTube-style educational videos. The system combines local Large Language Models (LLMs), state-of-the-art voice cloning (XTTS-v2), speech recognition (Whisper), and automated video editing (MoviePy/FFmpeg) into a streamlined, high-efficiency workflow.

2. System Architecture Overview

The system is structured as a modular content generation pipeline with three primary stages and a planned future API layer.

Component	Technology	Primary Function
LLM	Ollama + Qwen2.5	Script Generation and Stylization
Speech Recognition	Whisper	Style Analysis and Transcription
Voice Cloning	XTTS-v2	Long-Form Narration Synthesis
Talking Face	SadTalker + GFPGAN	Lip-Synced Avatar Animation
Video Editing	MoviePy + FFmpeg	Scene Segmentation, Timeline Building, and Rendering
Queue System (Future)	Celery + Redis	Asynchronous Job Management

2.1 Voice + Podcast Pipeline

This pipeline generates long-form narration audio from a cloned voice, starting with a YouTube voice sample. It involves audio cleanup, transcription, LLM script generation, text pre-processing, chunking, and final audio merging.

2.2 Talking Face Video Pipeline

This stage converts a static character image and the generated narration audio into a lip-synced video using facial animation and post-processing enhancement.

2.3 Scene-Based Video Editor

The main editor orchestrates the final video assembly, automatically segmenting the script into scenes (Introduction, Explanation, Demo, Outro). Using MoviePy and FFmpeg, it maps visual assets, overlays dynamic subtitles, and ensures precise audio-visual synchronization.

2.4 API Layer (Future Task)

A future layer will expose the pipeline via a REST API endpoint (*POST /generate-video*), managed by a Celery/Redis job queue to handle resource-intensive, asynchronous rendering.

3. Detailed Pipeline Implementation

3.1 Voice & Podcast Pipeline Steps

1. **YouTube Voice Sample:** Extracts audio using *yt-dlp* to serve as the voice reference.
2. **Audio Cleanup:** Normalizes and cleans the reference audio using FFmpeg. Processing includes mono conversion, sample rate normalization (*-ar 24000*), denoising (*afftdn*), and loudness normalization (*loudnorm*).
3. **Whisper Speech Analysis:** Transcribes the reference audio to capture and inject the speaker's style, tone, and pacing into the LLM prompt.
4. **LLM Script Generation:** A local LLM (Qwen2.5:7b via Ollama) generates a long educational script, explicitly formatted to sound natural when spoken.
5. **Text Cleaning and Formatting:** Cleans the script, replacing programming syntax with speech-friendly terms.
6. **Sentence Chunking:** Splits the script into smaller, safer chunks (e.g., *MAX_CHARS_PER_CHUNK = 220*) to prevent VRAM spikes and improve XTTS stability.
7. **XTTS-v2 Voice Cloning:** Generates cloned narration WAV files for each chunk using GPU acceleration.
8. **Audio Merge:** Concatenates all WAV chunks using FFmpeg's concat protocol.

3.2 Talking Face Video Pipeline Steps

1. **SadTalker Animation:** Generates lip-movement, facial motion, and eye blinking based on the narration audio.
2. **GFPGAN Face Enhancement:** Sharpens facial features and reduces artifacts to ensure high-quality results.

3.3 Scene-Based Video Editor Steps

1. **LLM Scene Script Generation:** LLM generates a structured script with explicit sections.
2. **Scene Segmentation:** Script is divided into distinct logical scenes.
3. **Asset Mapping:** Visual assets are mapped to their corresponding scenes.

- 4. **Timeline Builder:** MoviePy and FFmpeg arrange clips and transitions.
- 5. **Subtitle and Effects Engine:** Generates synchronized subtitles and dynamic effects.
- 6. **Audio-Video Synchronization:** Aligns narration with visual elements.
- 7. **Final Render:** Exports complete video as MP4.

4. Technical Implementation and Code Details

4.1 VRAM and Performance Optimization

The system includes aggressive VRAM management:

```
Python
# Disables gradient tracking to save VRAM
torch.set_grad_enabled(False)
# Prevents unnecessary GPU memory usage
torch.backends.cudnn.benchmark = False
# Frees unused CUDA memory
torch.cuda.empty_cache()
```

4.2 Configuration Parameters

Parameter	Value	Purpose
OLLAMA_MODEL	"qwen2.5:7b"	Defines the local LLM model.
MAX_REF_SECONDS	20	Limits reference audio length.
MAX_CHARS_PER_CHUNK	220	Controls TTS pass size.
TARGET_MINUTES	10	Estimates required script length.

4.3 TTS-Friendly Symbol Conversion

The function `make_tts_friendly()` ensures natural speech quality.

Symbol/Syntax	Spoken Conversion
==	"equals equals"
+	"plus"
/	"slash"
*	"times"

>=	"greater than or equal to"
!=	"not equals"

5. Roadmap & Future Integration

- Full MoviePy video rendering capabilities.
- Dynamic subtitle animation engine.
- FastAPI backend and Celery/Redis queue.

6. Hardware Requirements

Specification	Minimum	Recommended
GPU	RTX 3060 (12GB)	RTX 4070+
RAM	16GB	32GB

7. Conceptual Output Examples

- **Narration Audio:** High-quality WAV file.
- **Talking Avatar Video:** Enhanced MP4 video.
- **Final Edited Video:** Synchronized educational MP4.

 2026_05_22_13.08.45(1).mp4